SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0313	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	Athivaram Jayaprakash	**Reg. No.:**	192425311
Section / Batch:	AIML / 2024	**Date:**	

Code of Conduct

I **Athivaram Jayaprakash (192425311)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Athivaram Jayaprakash

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

QUESTION 1 – Resume Information Extraction

1. Problem understanding

A recruitment company receives many resumes in text format. Manually reading every resume takes time. The program uses Regular Expressions to identify a candidate's name, email, mobile number, technical skills and years of experience. It then creates a structured profile and checks whether the candidate has at least 2 years of experience and Python skill.

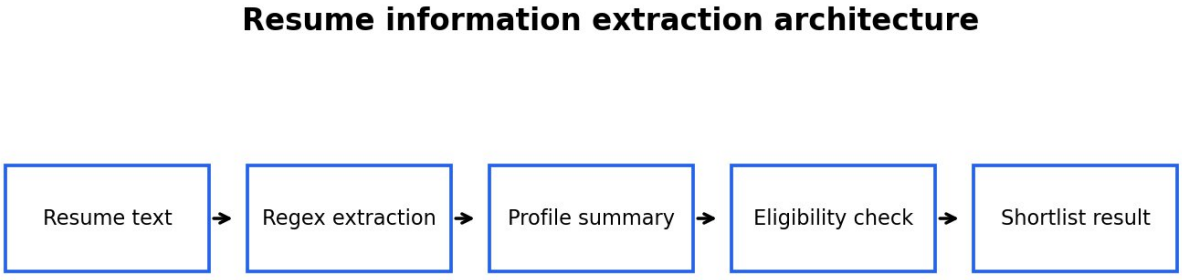
2. Aim

To develop a Python-based resume information extraction system using Regular Expressions and shortlist candidates based on the given eligibility conditions.

3. Modules

Resume input module – receives resume text.
Information extraction module – extracts name, email, mobile number, skills and experience.
Profile generation module – stores extracted details in a structured dictionary.
Eligibility module – checks minimum 2 years of experience and the presence of Python.

4. Architecture



5. Algorithm / steps

- Import the re module.
- Read the resume text.
- Use a labelled pattern to extract the candidate's name.
- Use an email pattern and a mobile-number pattern to find contact details.
- Compare the resume text with the required skill list: Python, Java, SQL, Machine Learning and NLP.
- Extract the number of years of experience.
- Store the extracted values in a dictionary and display the profile.
- Mark the candidate eligible only when experience is at least 2 years and Python is present.

6. Python program

CODE

```
import re
def extract_resume_details(text):
    # Extract name written after "Name:"
    name_match = re.search(r"(?im)^\s*Name\s*:\s*(.+)$", text)

    # Find email address
    email_match = re.search(
        r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b", text
    )

    # Find a 10-digit Indian mobile number, optionally with +91
    mobile_match = re.search(r"(?:\+91[\s-]?)?[6-9]\d{9}\b", text)

    # Detect only the skills listed in the question
    skill_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
    found_skills = [
        skill for skill in skill_list
        if re.search(r"\b" + re.escape(skill) + r"\b", text, re.I)
    ]

    # Extract years of experience
    exp_match = re.search(
        r"(\d+(?:\.\d+)?)\s*\+?\s*years?\s*(?:of\s*)?experience",
        text, re.I
    )
    years = float(exp_match.group(1)) if exp_match else 0

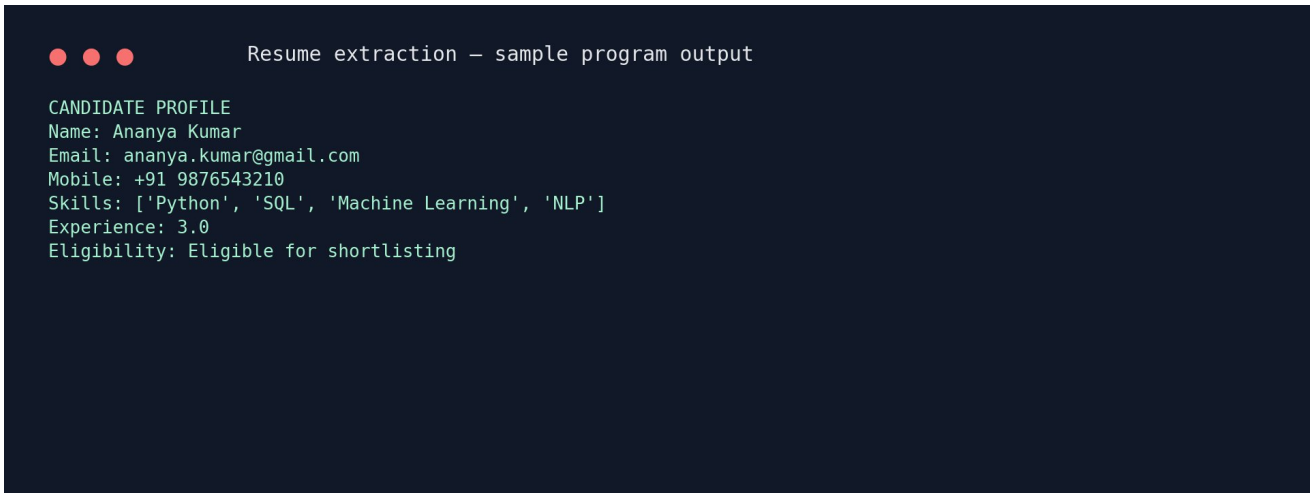
    return {
        "Name": name_match.group(1).strip() if name_match else "Not found",
        "Email": email_match.group(0) if email_match else "Not found",
        "Mobile": mobile_match.group(0) if mobile_match else "Not found",
        "Skills": found_skills,
        "Experience": years
    }

profile = extract_resume_details(resume)

print("CANDIDATE PROFILE")
for key, value in profile.items():
    print(f"{key}: {value}")

eligible = profile["Experience"] >= 2 and "Python" in profile["Skills"]
print("Eligibility:", "Eligible for shortlisting" if eligible
      else "Not eligible for shortlisting")
```

7. Output screenshot



8. Explanation of important code

import re: imports Python's Regular Expression library.
re.search(): searches the resume text and returns the first matching result.
re.I: makes skill and experience matching case-insensitive.
re.escape(skill): treats a skill name as literal text inside the pattern.
The dictionary profile stores the extracted information using field names.
The eligibility condition uses AND, so both requirements must be true.

9. Test cases

Input condition	Expected result
3 years, Python present	Eligible for shortlisting
1 year, Python present	Not eligible – experience is below 2 years
4 years, Python absent	Not eligible – Python skill is missing
No experience text	Experience is treated as 0; not eligible

10. Result / conclusion

The program extracts the required candidate information and creates a structured profile. It correctly shortlists the sample candidate because the candidate has 3 years of experience and knows Python. This is a basic rule-based extraction system; resumes with different layouts or unusual wording may need additional patterns.

QUESTION 2 – Regular Expression Product Search System

1. Problem understanding

An e-commerce website needs flexible product searching. The program searches a product catalogue using exact, prefix, suffix, partial and case-insensitive matching. It displays all matching product names and the number of matches for each search.

2. Aim

To implement a Python product search system using Regular Expressions for different keyword matching methods and generate a match-count report.

3. Modules

Product catalogue module – stores the available product names.
Search input module – receives the keyword and selected search type.
Regex pattern module – creates the pattern for the requested search operation.
Matching module – checks every product and collects matching names.
Report module – displays matching products and the total count.

4. Architecture

Product search system architecture



5. Search types

Search type	Meaning	Pattern idea
Exact	The complete product name must equal the keyword.	^keyword$
Prefix	The product name must begin with the keyword.	^keyword
Suffix	The product name must end with the keyword.	keyword$
Partial	The keyword may occur anywhere in the product name.	keyword
Case-insensitive	Matches letters regardless of uppercase/lowercase.	keyword with re.I

6. Algorithm / steps

- Import the re module and create a list of product names.
- Accept a keyword and search type.
- Remove extra spaces and escape the keyword using re.escape().
- Build the correct regex pattern based on the search type.
- Search every product name using re.search().
- For case-insensitive search, pass the re.I flag.
- Display the matching product names and calculate len(matches).
- Repeat the process for another search if required.

7. Python program

```
import re

products = [
    "Apple iPhone 15",
    "Samsung Galaxy S24",
    "Apple MacBook Air",
    "OnePlus Nord Buds",
    "Sony Wireless Headphones",
    "Dell Inspiron Laptop"
]

def search_products(keyword, search_type):
    keyword = keyword.strip()
    if not keyword:
        return []

    # Escape user input so special regex characters are treated literally
    safe_keyword = re.escape(keyword)

    if search_type == "exact":
        pattern = r"^" + safe_keyword + r"$"
    elif search_type == "prefix":
        pattern = r"^" + safe_keyword
    elif search_type == "suffix":
        pattern = safe_keyword + r"$"
    elif search_type == "partial":
        pattern = safe_keyword
    elif search_type == "case-insensitive":
        pattern = safe_keyword
    else:
        raise ValueError("Unknown search type")

    flags = re.I if search_type == "case-insensitive" else 0
    return [product for product in products
```

```
        if re.search(pattern, product, flags)]

# Demonstrate all five search operations
tests = [
    ("Apple iPhone 15", "exact"),
    ("Apple", "prefix"),
    ("Laptop", "suffix"),
    ("Galaxy", "partial"),
    ("apple", "case-insensitive")
]

for keyword, mode in tests:
    matches = search_products(keyword, mode)
    print(f"{mode.upper()} search for '{keyword}':")
    print("Matches:", matches if matches else "No matching products")
    print("Total matches:", len(matches))
    print("-" * 50)
```

8. Output screenshot



9. Explanation of important code

^keyword$ checks for an exact full-string match.
^keyword checks whether the text starts with the keyword.
keyword$ checks whether the text ends with the keyword.
A pattern containing only the keyword finds it anywhere in the product name.
re.I ignores uppercase/lowercase differences.
re.escape() prevents characters such as ., *, or + in a search keyword from acting as regex operators.
len(matches) gives the total number of matching products.

10. Test cases

Keyword	Search type	Expected count
Apple iPhone 15	Exact	1
Apple	Prefix	2
Laptop	Suffix	1
Galaxy	Partial	1
apple	Case-insensitive	2
Tablet	Partial	0

11. Result / conclusion

The product search program supports all five required search methods and displays matching product names with their counts. It improves flexibility compared with a simple case-sensitive string comparison. The sample tests show that case-insensitive search finds both Apple products even when the entered keyword is lowercase.

QUESTION 3 – University Registration Validation System

1. Problem understanding

A university registration portal must verify student information before course enrollment. The program validates the register number, institutional email, course code, semester and mobile number. It displays a message for each field and reports whether the complete registration is successful.

2. Aim

To build a Python validation system using Regular Expressions that checks student registration details and generates a final registration status.

3. Validation rules used in this example

The assessment question does not provide exact field formats. Therefore, the program uses the following illustrative rules. The institutional email domain is assumed to be saveetha.com; change this pattern if the university uses another official domain.

Field	Assumed rule	Regex pattern
Register number	Exactly 9 digits	^\d{9}$
Institutional email	Valid email username followed by @saveetha.com	^[A-Za-z0-9._%+-]+@saveetha\.com$
Course code	3 uppercase letters followed by 2 to 4 digits	^[A-Z]{3}\d{2,4}$
Semester	A single digit from 1 to 8	^[1-8]$
Mobile number	10 digits; first digit from 6 to 9	^[6-9]\d{9}$

4. Modules

Student input module – collects the student's registration details.
Field validation module – checks each field against its regex pattern.
Message module – displays Valid or Invalid for each field.
Registration status module – checks whether all fields are valid and generates the final result.

5. Architecture

University registration architecture



6. Algorithm / steps

- Import the re module.
- Define one regex pattern for each registration field.
- Collect the student details.
- Use re.fullmatch() to validate each complete field value.
- Display an individual validation message for every field.
- Store each validation result as True or False.
- Use all() to check whether every field is valid.
- Display SUCCESSFUL if all validations pass; otherwise display FAILED.

7. Python program

```
import re

# Assumptions: 9-digit register number; Saveetha email domain;
# course code = 3 uppercase letters + 2 to 4 digits;
# semester = 1 to 8; Indian mobile = 10 digits starting 6-9.
PATTERNS = {
    "register": r"^\d{9}$",
    "email": r"^[A-Za-z0-9._%+-]+@saveetha\.com$",
    "course": r"^[A-Z]{3}\d{2,4}$",
    "semester": r"^[1-8]$",
    "mobile": r"^[6-9]\d{9}$"
}

def validate(field, value):
```

```
    return re.fullmatch(PATTERNS[field], value.strip()) is not None

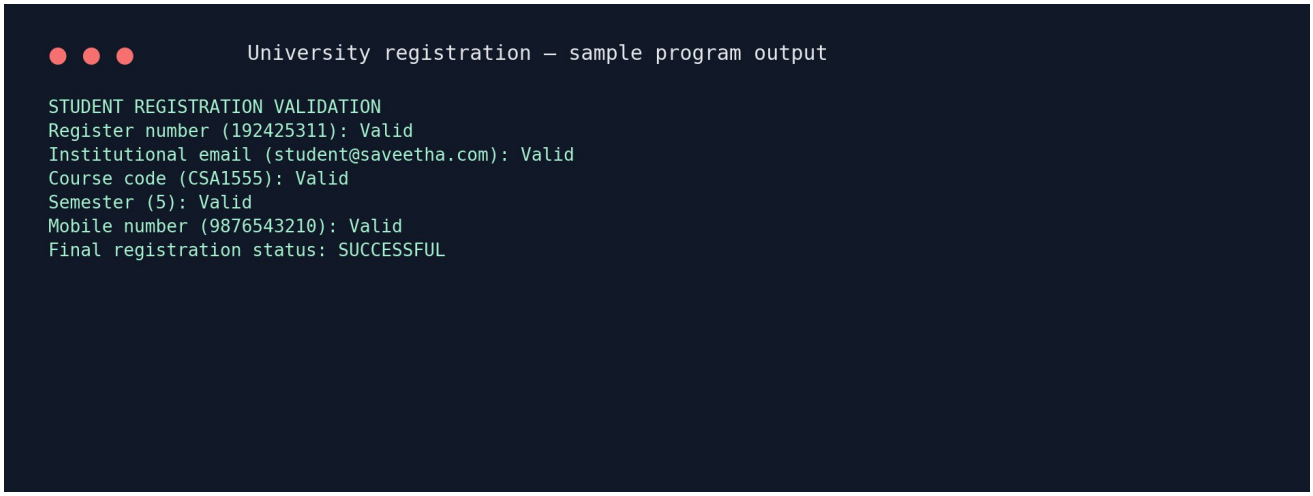
student = {
    "register": "192425311",
    "email": "student@saveetha.com",
    "course": "CSA1555",
    "semester": "5",
    "mobile": "9876543210"
}

labels = {
    "register": "Register number",
    "email": "Institutional email",
    "course": "Course code",
    "semester": "Semester",
    "mobile": "Mobile number"
}

results = {}
print("STUDENT REGISTRATION VALIDATION")
for field, value in student.items():
    is_valid = validate(field, value)
    results[field] = is_valid
    message = "Valid" if is_valid else "Invalid"
    print(f"{labels[field]} ({value}): {message}")

registration_success = all(results.values())
print("Final registration status:",
      "SUCCESSFUL" if registration_success else "FAILED")
```

8. Output screenshot



9. Explanation of important code

PATTERNS stores the validation rule for each field.
re.fullmatch() validates the entire input, not just a part of it.
^ and $ mark the beginning and end of a pattern.
\d{9} means exactly nine digits.
[A-Z]{3} means exactly three uppercase letters.
[1-8] accepts semester values from 1 through 8.
all(results.values()) returns True only when every field is valid.

10. Test cases

Test input	Expected validation
192425311	Valid register number
19242A311	Invalid register number – contains a letter
student@gmail.com	Invalid institutional email under the assumed domain rule
CSA1555	Valid course code
CS1555	Invalid course code – only two letters
9	Invalid semester under the 1–8 rule
9876543210	Valid mobile number
12345	Invalid mobile number – not 10 digits

11. Result / conclusion

The program validates all five student details and displays a field-wise report. When every field follows the assumed format, the final status is SUCCESSFUL. If at least one field is invalid, the final status becomes FAILED. In a real university portal, the regex rules must be aligned with the official student-data format.